Experiment 5

Title: Hash Table Implementation with Chaining for Collision Resolution

Objectives:

To understand the concept of hash tables and their applications.

To implement a hash table using the division method as a hash function.

To implement chaining as a collision resolution technique.

To implement essential hash table operations: Insert, Search, and Delete.

Problem Statement:

Develop a hash table data structure of size 10 that utilizes the division method for hashing keys. In the event of a collision, implement chaining to store multiple elements at the same index. The implementation must support insertion of key-value pairs, searching for values based on keys, and deletion of key-value pairs.

Outcomes:

A functional hash table implementation capable of storing, retrieving, and deleting data efficiently.

A clear understanding of hashing, collision resolution (chaining), and fundamental hash table operations.

Proficiency in implementing data structures in a chosen programming language.

Software and Hardware Requirements:

Software:

Operating System: Windows 10/11, macOS, or Linux distribution.

Programming Language: Python 3.x, Java Development Kit (JDK), C++ compiler (e.g., GCC), etc.

Integrated Development Environment (IDE) or Text Editor: VS Code, IntelliJ IDEA, Eclipse, PyCharm, etc.

Hardware:

Processor: Intel Core i3 or equivalent.

RAM: 4GB or more.

Storage: Minimum 100MB free space.

Theory Concept in Brief:

A hash table is a data structure that maps keys to values using a hash function to compute an index, also known as a hash code, into an array of buckets or slots. The division method calculates the hash index as key % table_size. Chaining is a collision resolution technique where each bucket in the hash table points to a linked list (or similar data structure) containing all key-value pairs that hash to that same index. This allows multiple elements to be stored at the same hash index without overwriting existing data.

Algorithm:

Hash Table Structure:

An array of size 10, where each element is a reference to a linked list (or list in Python).

Hash Function (Division Method):

hash_index = key % 10

Insert(key, value):

Calculate hash_index = key % 10.

If the linked list at hash_index is empty, create a new linked list and add the (key, value) pair.

Otherwise, traverse the linked list at hash_index. If key already exists, update its value. Else, add the (key, value) pair to the end of the linked list.

Search(key):

Calculate hash_index = key % 10.

Traverse the linked list at hash_index.

If key is found, return its associated value.

If key is not found after traversing the entire list, return an indication of not found (e.g., None).

Delete(key):

Calculate hash_index = key % 10.

Traverse the linked list at hash_index.

If key is found, remove the (key, value) pair from the linked list.

If key is not found, indicate that the key was not present.

Flowchart:

Test Cases:

Insert:

Insert (5, "Apple"), (15, "Banana"), (25, "Cherry") - all hash to index 5.

Insert (3, "Dog"), (13, "Elephant") - all hash to index 3.

Search:

Search for "Apple" (key 5). Expected: "Apple".

Search for "Cherry" (key 25). Expected: "Cherry".

Search for "Cat" (key 7). Expected: Not found.

Delete:

Delete "Banana" (key 15). Expected: "Banana" removed from index 5.

Delete "Dog" (key 3). Expected: "Dog" removed from index 3.

Delete "Zebra" (key 99). Expected: Key not found.

Test Data Set (if applicable):

Code

Key, Value 5, Apple 15, Banana 25, Cherry 3, Dog 13, Elephant 7, Grape 17, Fig

Mathematical Model (if applicable):

Hash Function:

h(k)=k(modm) ,

where is the key

■ is the table size (10 in this case).

Load Factor:

α=n/m,where

■ is the number of elements and mm

■ is the table size. In chaining,

■ can be greater than 1.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates a hash table using the division method and chaining for collision resolution. The Insert, Search, and Delete operations are functional, providing efficient data storage and retrieval. Chaining effectively handles collisions, preventing data loss and maintaining the integrity of the hash table. Performance for these operations is generally O(1)cap O open paren 1 close paren ■(1) on average, but can degrade to O(n)cap O open paren n close paren ■(■)in the worst-case scenario where all elements hash to the same bucket, resulting in a long linked list traversal. Optimizing the hash function and table size can mitigate this performance degradation.